

6.0 IMPLEMENTATION DIAGRAM

Implementation diagrams are used to model the physical environment of a system and how its components will be deployed.

There are two main types of implementation diagrams:

- i. Structural diagrams
- ii. Behavioral diagrams

Structural diagrams: are used to describe the static structure of a system. They include the following diagrams; Class Diagram, package diagram, object diagram, component diagram, composite structure diagram and deployment diagram

Behavioral diagrams: are used to describe the dynamic behavior of a system. They include the following diagrams; Activity diagram, use case diagram, timing diagram, state machine diagram, communication diagram and sequence diagram.

6.1 COMPONENT DIAGRAM

A component diagram is a type of Unified Modeling Language (UML) diagram that depicts how components are wired together to form larger components or software systems. Component diagrams are essentially class diagrams that focus on a system's components and are used to model the static implementation view of a system.

6.2 USES OF COMPONENT DIAGRAM

Some of the uses of component diagrams include:

- i. Visualizing the components of a system
- ii. Describing the organization and relationships among components in a system
- iii. Modeling the static implementation view of a system
- iv. Helping to build executable systems through forward and reverse engineering

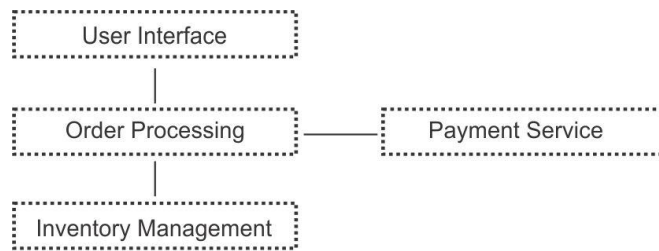
6.3 PROCESS OF MODELING WITH COMPONENT DIAGRAM

Here's a step-by-step process for modeling with a component diagram:

1. Identify Components: Begin by identifying the main components of your system. They can be software packages, web services, databases, or other system parts.
2. Define Interfaces: For each component, define the interfaces it provides and requires. Interfaces represent the points of interaction with other components.
3. Establish Dependencies: Determine the dependencies between components. Dependencies show how components rely on each other to function. These are usually represented with dashed arrows pointing from the dependent component to the component it depends on.
4. Draw the Diagram: Use a UML tool or diagram draw your component diagram. Represent each component as a rectangle with its name inside. Draw interfaces as small circles or sockets on the edge of the component. Use dashed arrows to show dependencies between components.
5. Validate the Diagram: Review your component diagram to ensure it accurately represents the system's architecture.

Example Component Diagram

Here's a simple example of a component diagram for an online shopping system:



Component Diagram

Fig 4. Component Diagram

In this diagram:

- 'User Interface' is a component that provides an interface for users to interact with the system.
- 'Order Processing' is a component that handles order processing. It depends on the 'Inventory Management' component for checking product availability and the 'Payment Service' component for processing payments.
- 'Payment Service' is a component that processes payments.
- 'Inventory Management' is a component that manages product inventories.

6.4 DEPLOYMENT DIAGRAM

A deployment diagram is a type of structural diagram in the Unified Modeling Language (UML) that models the physical deployment of artifacts on nodes. It is used to describe the hardware components where software components are deployed. A deployment diagram shows the configuration of run time processing nodes and the components that live on them. It is used to visualize the topology of the physical components of a system, where the software components are deployed.

6.5 PURPOSE OF DEPLOYMENT DIAGRAM

- i. To represent how software is installed and interact with hardware component
- ii. To visualize the topology of the physical component where software is deployed
- iii. Deployment diagram important for system engineers as they help control parameters such as performance, scalability, maintainability, and portability

6.6 PROCESS OF MODELING WITH DEPLOYMENT DIAGRAM

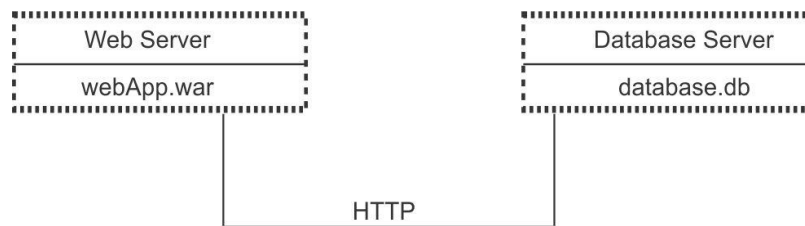
Here's how to model with a deployment diagram step-by-step:

1. Identify Nodes: Nodes represent physical devices or execution environments (e.g., servers, workstations, mobile devices). Identify all the nodes that will be part of your system.

2. Define Artifacts: Artifacts are the physical pieces of information that are deployed on nodes (e.g., executables, libraries, databases). Identify the artifacts that need to be deployed within your system.
3. Establish Relationships: Determine the relationships between nodes and artifacts. Artifacts are deployed on nodes, and there can be communication between nodes.
4. Draw the Diagram: Use a UML tool or diagramming software to draw your deployment diagram. Represent each node as a three-dimensional box. Inside the nodes, list the artifacts deployed on them. Connect nodes with lines to indicate communication paths.
5. Validate the Diagram: Review your deployment diagram to ensure it accurately represents the system's physical deployment. Validate the diagram with stakeholders and make necessary adjustments.

Example Deployment Diagram

Here's a simple example of a deployment diagram for a web application:



Deployment Diagram

Fig. 5. Deployment Diagram

In this diagram:

- `Web Server` is a node representing the server where the web application is deployed.
- `Database Server` is a node representing the server where the database is hosted.
- `webApp.war` is an artifact deployed on the web server.
- `database.db` is an artifact deployed on the database server.
- There is an HTTP communication path between the web server and the database server.

7.0 USE CASE DIAGRAM

A use case diagram is a graphical representation of the interactions between a user and a system. It is a way to summarize the details of a system and the users within that system.

The main components/elements of a use case diagram include

- i. The boundary
 - ii. Actors
 - iii. Use Cases
 - iv. The relationships between and among the actors and the use cases.
- The boundary defines the system of interest in relation to the world around it.
 - The actors are individuals involved with the system defined according to their roles
 - The use cases are the specific roles played by the actors within and around the system

- The relationships between and among the actors and the use cases are depicted using specialized symbols and connectors.

There are **three types** of relationships between actors and use cases:

Association, Generalization, and include.

7.1 USES OF USE CASE DIAGRAM

The following are some of the uses of a use case diagram:

- Represent the goals of systems and users
- Specify the context a system should be viewed in
- Specify system requirements
- Provide a model for the flow of events when it comes to user interactions
- Provide an outside view of a system
- Show external and internal influences on a system

7.2 TYPES OF USE CASE DIAGRAM

Use case diagrams are classified into two types:

- Behavioral UML Diagrams
- Structural UML diagrams

Behavioral UML diagrams provide a visual representation of the dynamic behavior of a system, while structural UML diagrams show the static structure of a system.

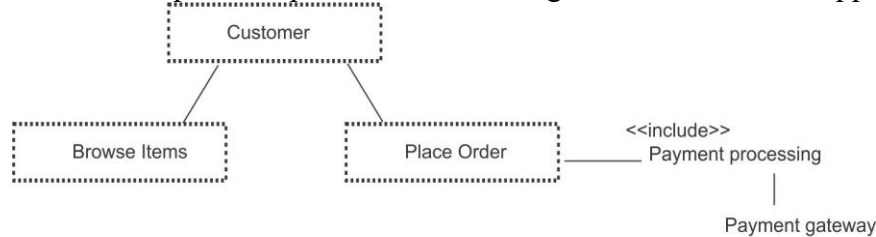
7.3 PROCESS OF MODELING WITH USE CASE

Here's how to model with a use case diagram step-by-step:

1. Identify Actors: Actors represent entities that interact with the system. These can be human users, other systems, or hardware devices. Identify all the actors involved in your system.
2. Define Use Cases: Use cases describe the functionalities or services that the system provides to the actors. Identify and define all the use cases for the system. Each use case should represent a distinct function or interaction.
3. Establish Relationships: Determine the relationships between actors and use cases. Common types of relationships include:
 - a. Association: Represents communication between an actor and a use case.
 - b. Include: Shows that a use case uses the behavior of another use case.
 - c. Extend: Represents optional behavior that extends a use case.
 - d. Generalization: Indicates inheritance between actors or use cases.
4. Draw the Diagram: Use a UML tool or diagramming software to draw your use case diagram. Represent each actor as a stick figure and each use case as an oval. Connect actors to use cases with lines to show associations. Use dashed arrows to indicate include and extend relationships.
5. Validate the Diagram: Review your use case diagram to ensure it accurately represents the system's functional requirements. Validate the diagram with stakeholders and make necessary adjustments.

Example Use Case Diagram

Here's a simple example of a use case diagram for an online shopping system:



Usecase diagram

In this diagram:

- Actors: The `Customer` represents a user interacting with the system.
- Use Cases: The main use cases are `Browse Items` and `Place an Order`.
- Include Relationship: `Place an Order` includes `Payment Processing`.
- Extend Relationship: Optional behavior can be added to extend specific use cases.

8.0 ACTIVITY DIAGRAM

An activity diagram is a type of flowchart that shows the flow of activities or actions in a system or process. It is a behavioral diagram in the Unified Modeling Language (UML) that captures the dynamic behavior of a system.

8.1 USES OF ACTIVITY DIAGRAM

- Activity diagrams can be used to model workflows
- Activity diagrams can also describe the steps in a use case diagram.
- Activity diagrams are used in software engineering to understand the flow of programs at a high level.
- Activity diagrams can be used to show customer journeys, such as the process of a user entering their credentials and logging in to a banking app

8.2 PROCESS OF MODELING WITH ACTIVITY DIAGRAM

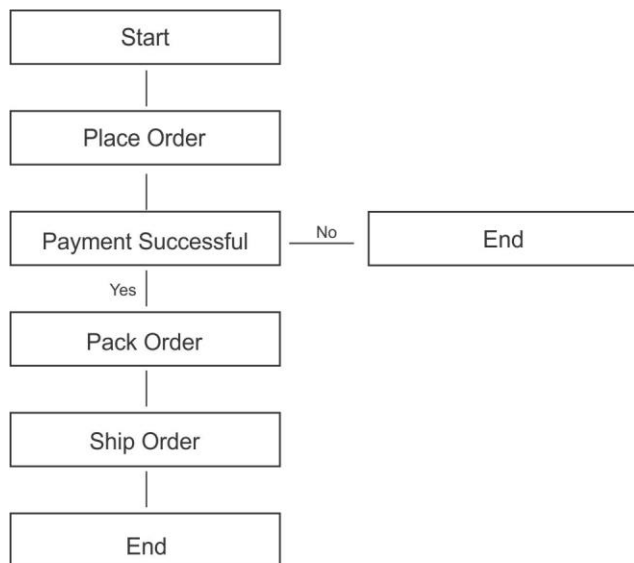
To model a system or process using an activity diagram, you can follow these steps:

1. Identify Activities: Start by identifying the main activities that need to be represented. An activity is a task or function that is performed within the system.
2. Define Actions: Break down each activity into specific actions. Actions are the individual steps or operations that make up an activity.
3. Establish Flow: Determine the sequence of activities and actions. Identify how one action leads to the next and any conditions or decisions that affect the flow.
4. Identify Decision Points: Determine where decisions need to be made in the workflow. Decision points are represented as diamonds in the diagram and show where different paths can be taken based on conditions.
5. Add Control Flows: Control flows are arrows that show the direction of the flow between activities, actions, and decision points. Draw arrows to connect the elements in the correct sequence.

6. Include Start and End Points: Start and end points indicate where the process begins and ends. The start point is represented by a filled circle, and the end point is represented by a filled circle within a larger circle.
7. Draw the Diagram: Use a UML tool or diagramming software to draw your activity diagram. Represent activities and actions as rounded rectangles, decision points as diamonds, and use arrows for control flows. Mark the start and end points clearly.
8. Validate the Diagram: Review your activity diagram to ensure it accurately represents the workflow. Validate the diagram with stakeholders and make any necessary adjustments.

Example Activity Diagram

Here's a simple example of an activity diagram for an online order process:



Activity diagram

Fig. 7. Activity Diagram